

PHARMACEUTICAL IMPORT & DISTRIBUTION MANAGEMENT SYSTEM

FINAL PROJECT REPORT

GROUP NO 13

AS2023531- K V S P WICKRAMARACHCHI
AS2023475- I M RATHNAYAKE
AS2023608- M W WIJEWARDANA
AS2023414- G M A I LIYANAGE
AS2023633- A D N METHMAL
AS2023417- G K LADDUWAHETTI
AS2023631- S A C HANSIKA



1 .Phase I - Project Proposal

Background

Pharmaceutical import companies in Sri Lanka act as critical intermediaries between international manufacturers and local healthcare providers (hospitals and pharmacies). Currently, most of these companies rely on manual or semi-digital processes. Orders from clients are received via phone calls, WhatsApp messages, or paper forms, which are then manually entered into spreadsheets. Inventory tracking is often done using basic software or even physical logbooks, making it difficult to track batch-wise stock, expiry dates, and supplier details in real time. The procurement process with foreign suppliers is similarly disconnected, leading to delays, stock inconsistencies, and operational inefficiencies. There is a clear need for an integrated digital system that can automate and streamline the entire supply chain, from client order placement to supplier procurement.

Problem Statement

The main problem is the lack of an integrated computer system. All operations are managed manually or through disconnected files, leading to critical inefficiencies:

1. **Inefficient Order Management:** Orders from hospitals/pharmacies are taken by phone or paper. This leads to wrong entries, duplicate orders, and slow processing.
2. **Poor Inventory Visibility:** The company does not know exactly what medicines are available right now. This causes two problems:
 - Running out of popular medicines (stockouts).
 - Having too much slow-selling medicines, which can expire and be wasted.
3. **Slow Procurement Process:** The process to order new stock from foreign companies is slow and done with papers. It is not connected to the sales data, so they often order too late.
4. **Lack of Client Self-Service:** Hospitals and pharmacies cannot check stock, place orders online, or see their past orders. They must always call or email, which takes a lot of time for the staff.
5. **Data Silos & Reporting Difficulties:** Sales info, stock details, and supplier data are kept in different Excel files and notebooks. Because of this, making useful reports or smart business decisions is very hard.

Therefore, to address these challenges effectively, there is a critical need to develop a formal, centralized database system integrated with a dedicated online platform.

Objectives

The primary goal of this project is to design and propose a centralized database system integrated with a dedicated online platform that addresses the above problems. The specific objectives are:

1. To develop a computerized system that can manage medicine orders, inventory, and supplier purchases in an efficient and accurate way.
2. To provide real-time information about medicine stock levels, batch expiry dates, and order status to both company staff and client hospitals/pharmacies.
3. To track all sales and procurement data in a centralized database, allowing the management to generate useful reports and make better

Scope

A. Functional Scope

The system will provide the following key functions ,

1. User Management & Authentication:
 - o Allow new Hospitals and Pharmacies to register online as Clients.
 - o Allow System Administrators to manage all user accounts.
 - o Authenticate users and control access based on their role (Admin, Hospital, Pharmacy).
2. Medicine & Inventory Management:
 - o Add, update, and delete medicine details (Medicine Master List).
 - o Display real-time stock availability to Clients.
 - o Generate automatic low-stock alerts for Administrators.
3. Sales Order Processing:
 - o Allow Clients to browse the available medicine catalog.
 - o Allow Clients to create Sales Orders by selecting items and quantities.
 - o Allow Clients to view their order history and track current order status.
 - o Automatically update stock levels when an order is approved.
 - o Allow Admins to view and update order status (Pending, Approved, Dispatched).

4. Procurement Management:

- Allow Admins to view low-stock reports.
- Allow Admins to create Purchase Orders for selected Suppliers.
- Maintain a complete record of all Purchase Orders and their status.
- Automatically update stock levels when a Purchase order is Received .

5. Basic Reporting:

- Generate essential reports: Low-stock reports, Sales Summaries, Expired Batches List, and Pending Orders Report.

2 .Phase II

2.1 Assumptions for Phase II

Procurement & Suppliers

As per Proposal: The system aimed to allow Admins to create Purchase Orders for selected Suppliers and automatically update stock levels upon receiving orders.

Assumption: It is assumed that the **primary goal of the system is to manage Sales Orders only**. The procurement process happens outside the system (Offline), and the Administrator updates the stock levels in the system.

Medicine & Inventory Management

As per Proposal: The system was intended to record batch-wise stock entries, including expiry dates and import costs, using separate entities for Medicine and Batch.

Assumption: To minimize database complexity, it is assumed that the system tracks only the "Current Active Batch" for each medicine. New batch details (Expiry Date, Quantity) are updated only after the existing stock is fully exhausted.

System Administration & Admin Role

As per Proposal: The system was to provide features for System Administrators to manage user accounts and procurement.

Assumption: It is assumed that the system is managed by a single designated administrator with pre-defined credentials. Since the administrator acts as the backend manager for identifying requirements and updating orders, this is treated as a direct system function rather than a separate database entity.

Client Classification

As per Proposal: The system intended to allow new Hospitals and Pharmacies to register online as Clients.

Assumption: It is assumed that every registered client must be either a Hospital or a Pharmacy (Total Specialization) and cannot be both at the same time (Disjoint Constraint). Shared information like name and address are stored in the Client table.

2.2 Identifying Entities

During the design phase, several key entities were identified to represent the core data of the Pharmaceutical Import and Distribution Management System. These entities were selected based on the requirements of the system.

- **CLIENT:** This was identified as the main entity to represent all customers (hospitals and pharmacies).
- **HOSPITAL & PHARMACY:** These were identified as specialized sub-type entities of the CLIENT super-type to store category-specific data.
- **MEDICINE_STOCK:** This entity was identified to manage the inventory and track medicine details.
- **SALES_ORDER:** This was identified as the entity to record transactions placed by clients.
- **SALES_ITEM:** This was identified as a weak entity, as its existence was determined to be dependent on the SALES_ORDER entity.

2.3 Identifying Attributes

Once the entities were established, specific attributes were identified for each to define their characteristics. Primary keys were also assigned to ensure each record was uniquely identified.

- **CLIENT:** Attributes such as Client_Id (Primary Key), Username, Password, Email, Phone_No, and Address (composed of Building_No, Street, and City) were assigned. A Client_Type attribute was also included to distinguish between hospitals and pharmacies.
- **HOSPITAL:** Specific attributes including Hospital_Name, Branch, Director_Name, and Ward_Count were identified.
- **PHARMACY:** Attributes such as Pharmacy_Name and Pharmacist_RegNo were assigned to this entity.
- **MEDICINE_STOCK:** This entity was defined with attributes including Medicine_Id (Primary Key), Generic_Name, Brand_Name, Dosage, Unit_Price, Expiry_Date, and Current_Quantity.
- **SALES_ORDER:** Attributes such as SalesOrder_Id (Primary Key), Order_Date, Total_Amount, and Status were identified. A Client_ID was included as a foreign key to link the order to a client.
- **SALES_ITEM:** For this weak entity, Quantity and Unit_price were identified as attributes. A composite primary key, consisting of SalesOrder_Id and Medicine_Id, was assigned.

2.4 Identifying Relationships

1. The "Places" Relationship :

A relationship was established between the CLIENT and SALES_ORDER entities.

It was determined that one client could place many sales orders, representing a 1:N (One-to-Many) cardinality.

Total participation was assigned to the SALES_ORDER side, as every order was required to be associated with a specific client.

2. The "Contains" Relationship (Identifying Relationship) :

A relationship was created between SALES_ORDER and the weak entity SALES_ITEM.

It was specified that a single sales order could contain multiple sales items (1:N).

This was defined as an identifying relationship, meaning a SALES_ITEM could not be uniquely identified without its parent SALES_ORDER.

3. The "Included_In" Relationship :

A connection was formed between MEDICINE_STOCK and SALES_ITEM.

It was determined that one type of medicine could be included in many different sales items across various orders (1:N).

2.5 Specialization

During the design of the CLIENT entity, a specialization approach was adopted to manage the different types of customers in the system.

Reason for Specialization:

- It was observed that while all customers shared common attributes (like Address and Email), Hospitals and Pharmacies also possessed unique data fields.
- To maintain a clean database structure, the CLIENT entity was established as a super-class, while HOSPITAL and PHARMACY were created as sub-classes.

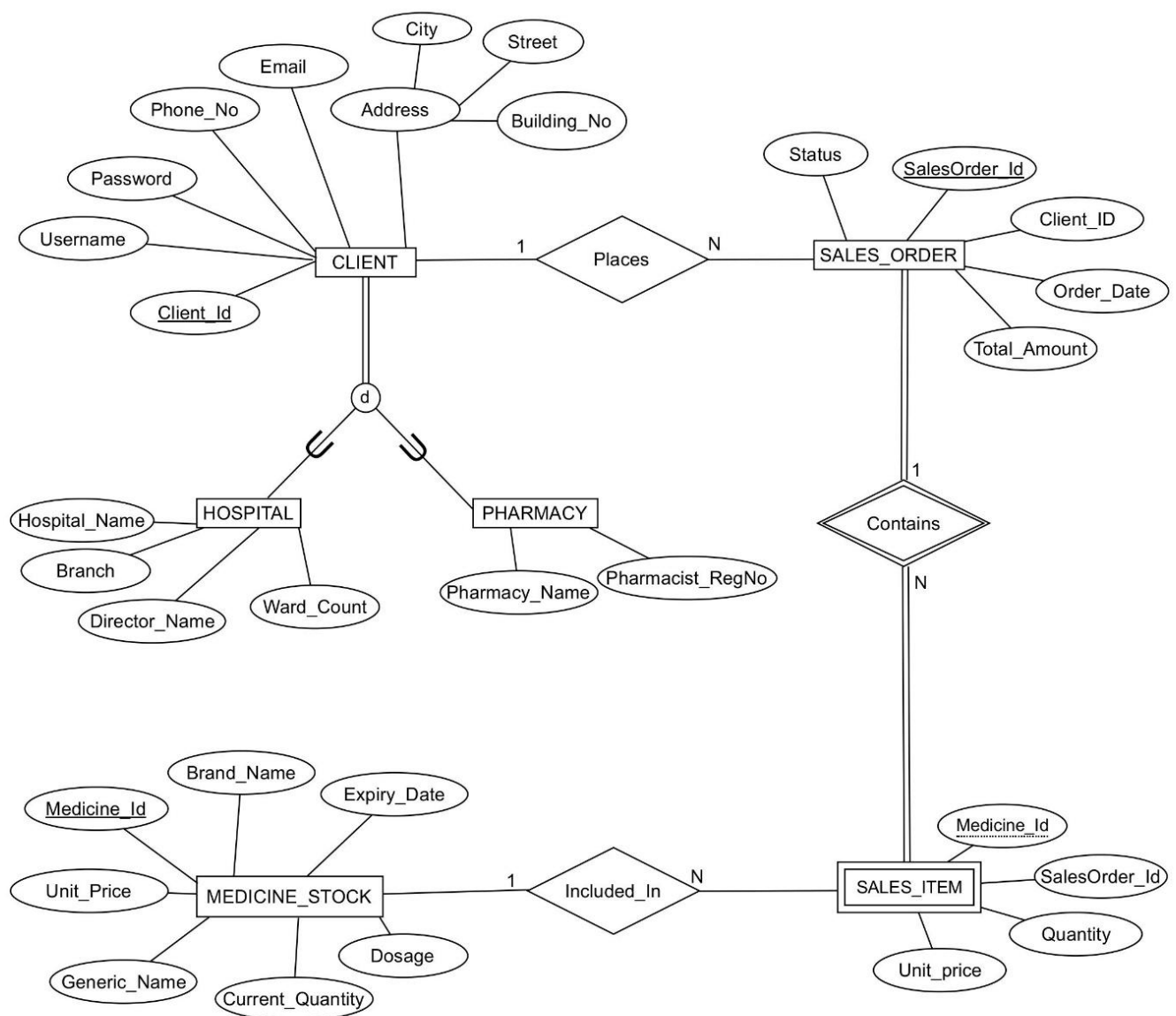
Disjoint Constraint (d):

- A disjoint constraint was applied to the relationship.
- This was decided to ensure that a single client record could only belong to one sub-type at a time.
- Therefore, it was programmed that a registered entity could be either a hospital or a Pharmacy, but it was prevented from being both simultaneously.

Total Specialization:

- Total specialization was enforced, which was represented by the double lines in the EER diagram.
- It was required that every client existing in the database must belong to at least one of the two sub-classes.

2.6 Enhanced Entity Relationship Diagram



2.7 Relational Schema

CLIENT (Client_Id , Username , Password , Email , Phone_No , Building_No , Street , City , Client_Type)

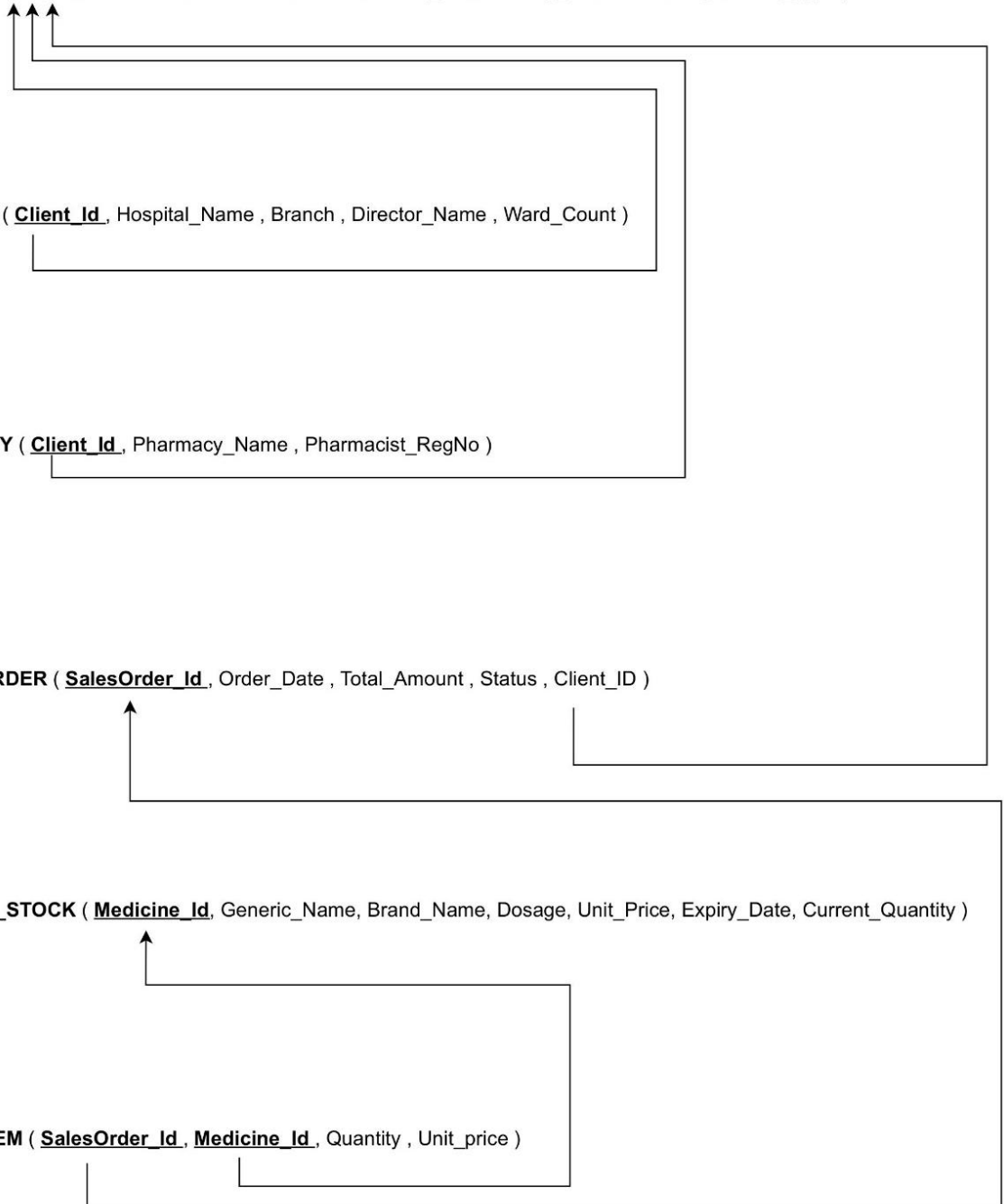
HOSPITAL (Client_Id , Hospital_Name , Branch , Director_Name , Ward_Count)

PHARMACY (Client_Id , Pharmacy_Name , Pharmacist_RegNo)

SALES_ORDER (SalesOrder_Id , Order_Date , Total_Amount , Status , Client_ID)

MEDICINE_STOCK (Medicine_Id , Generic_Name , Brand_Name , Dosage , Unit_Price , Expiry_Date , Current_Quantity)

SALES_ITEM (SalesOrder_Id , Medicine_Id , Quantity , Unit_price)

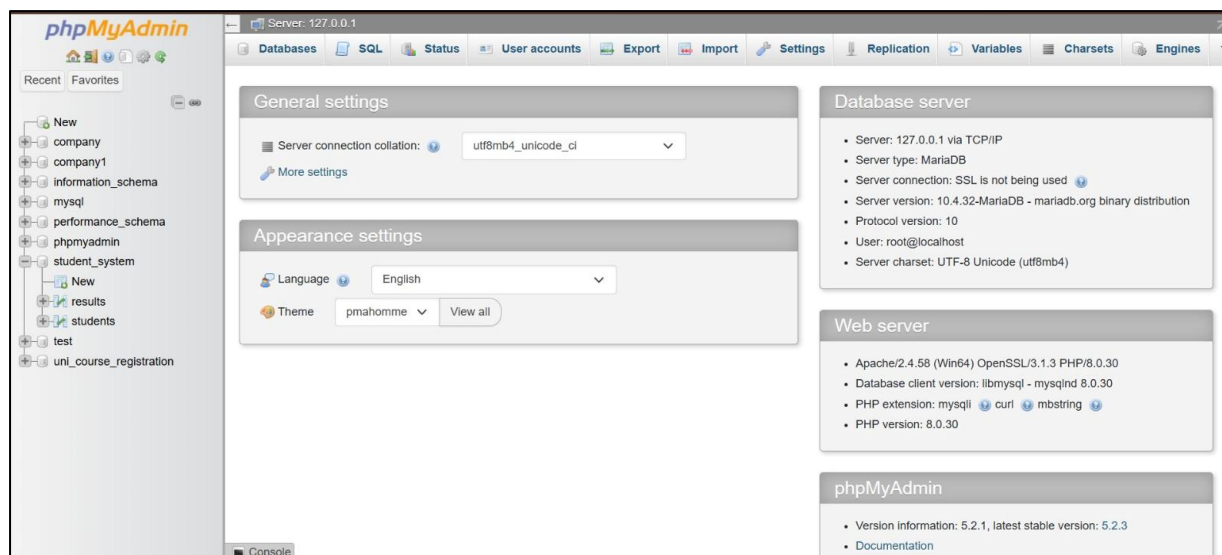


3. Phase III - Implementation

3.1 Database Implementation (SQL Queries)

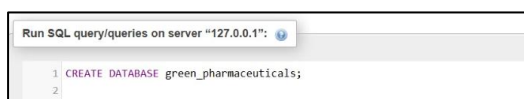
Database & Table Creation

The database schema was physically implemented using MySQL scripts in the phpMyAdmin environment. Several constraints and data types were utilized to ensure data integrity and security.

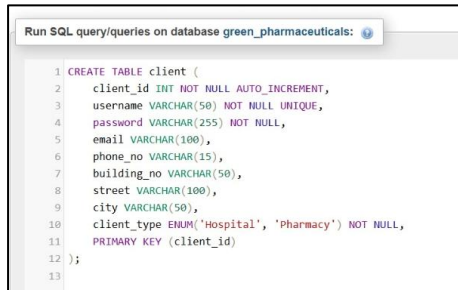


1. Database Creation

A dedicated database named `green_pharmaceuticals` was created to host the related tables of the system.



2. Client Table Implementation



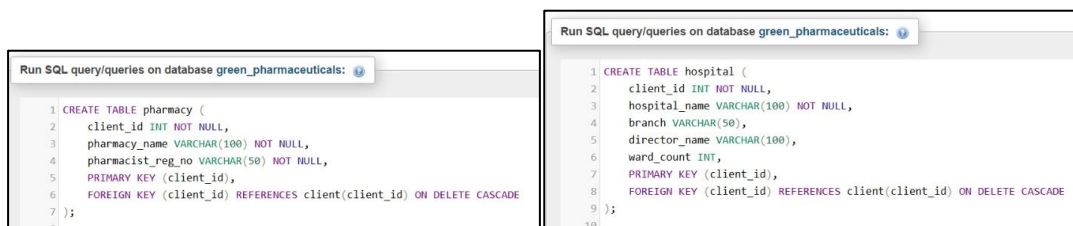
```
Run SQL query/queries on database green_pharmaceuticals: ⓘ
1 CREATE TABLE client (
2   client_id INT NOT NULL AUTO_INCREMENT,
3   username VARCHAR(50) NOT NULL UNIQUE,
4   password VARCHAR(255) NOT NULL,
5   email VARCHAR(100),
6   phone_no VARCHAR(15),
7   building_no VARCHAR(50),
8   street VARCHAR(100),
9   city VARCHAR(50),
10  client_type ENUM('Hospital', 'Pharmacy') NOT NULL,
11  PRIMARY KEY (client_id)
12 );
13
```

The client table was established as the central repository for all system users.

- The `client_id` was defined with the `NOT NULL AUTO_INCREMENT` constraint to ensure that every new client was automatically assigned a unique identification number that could not be empty.
- The `username` attribute was set as `NOT NULL UNIQUE` to prevent the creation of duplicate accounts.
- An `ENUM` data type was applied to the `client_type` column to strictly limit the input values to either 'Hospital' or 'Pharmacy', ensuring that no other invalid categories were entered.

3. Hospital and Pharmacy Tables (Specialization)

These tables were created to store category-specific data.



```
Run SQL query/queries on database green_pharmaceuticals: ⓘ
1 CREATE TABLE pharmacy (
2   client_id INT NOT NULL,
3   pharmacy_name VARCHAR(100) NOT NULL,
4   pharmacist_reg_no VARCHAR(50) NOT NULL,
5   PRIMARY KEY (client_id),
6   FOREIGN KEY (client_id) REFERENCES client(client_id) ON DELETE CASCADE
7 );
8

Run SQL query/queries on database green_pharmaceuticals: ⓘ
1 CREATE TABLE hospital (
2   client_id INT NOT NULL,
3   hospital_name VARCHAR(100) NOT NULL,
4   branch VARCHAR(50),
5   director_name VARCHAR(100),
6   ward_count INT,
7   PRIMARY KEY (client_id),
8   FOREIGN KEY (client_id) REFERENCES client(client_id) ON DELETE CASCADE
9 );
10
```

- In both tables, the `client_id` was established as a `FOREIGN KEY` that referenced the primary key of the client table.
- The `ON DELETE CASCADE` action was specifically implemented for these foreign keys. This ensured that if a client record was removed from the parent client table, all associated data in the hospital or pharmacy tables were automatically deleted by the system to maintain referential integrity.

4. Medicine Stock Table Implementation

The medicine_stock table was implemented to manage inventory records.

```
Run SQL query/queries on database green_pharmaceuticals:
1 CREATE TABLE medicine_stock (
2   medicine_id INT NOT NULL AUTO_INCREMENT,
3   generic_name VARCHAR(100) NOT NULL,
4   brand_name VARCHAR(100) NOT NULL,
5   dosage VARCHAR(50),
6   unit_price DECIMAL(10, 2) NOT NULL,
7   expiry_date DATE NOT NULL,
8   current_quantity INT NOT NULL DEFAULT 0,
9   PRIMARY KEY (medicine_id)
10 );
```

- The medicine_id was set to NOT NULL AUTO_INCREMENT for easy identification.
- A DECIMAL(10,2) data type was used for the unit_price to store financial values accurately.
- The current_quantity was configured with a DEFAULT 0 value to ensure that new records started with a zero balance if no quantity was specified.

5. Sales Order Table Implementation

The sales_order table was established to record transactions.

```
Run SQL query/queries on database green_pharmaceuticals:
1 CREATE TABLE sales_order (
2   sales_order_id INT NOT NULL AUTO_INCREMENT,
3   order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
4   total_amount DECIMAL(15, 2) DEFAULT 0.00,
5   status ENUM('Pending', 'Approved', 'Dispatched', 'Cancelled') DEFAULT 'Pending' NOT NULL,
6   client_id INT NOT NULL,
7   PRIMARY KEY (sales_order_id),
8   FOREIGN KEY (client_id) REFERENCES client(client_id)
9 );
10
```

- The order_date was configured with a DEFAULT CURRENT_TIMESTAMP to automatically record the exact time an order was placed.
- An ENUM was utilized for the status column to restrict order states to 'Pending', 'Approved', 'Dispatched', or 'Cancelled'.
- A FOREIGN KEY was added to link each order to a specific client in the client table.

6. Sales Item Table Implementation

The sales_item table was created to represent the weak entity that tracks individual medicines within an order.

```
Run SQL query/queries on database green_pharmaceuticals:

1 CREATE TABLE sales_item (
2   sales_order_id INT NOT NULL,
3   medicine_id INT NOT NULL,
4   quantity INT NOT NULL,
5   unit_price DECIMAL(10, 2) NOT NULL,
6   PRIMARY KEY (sales_order_id, medicine_id),
7   FOREIGN KEY (sales_order_id) REFERENCES sales_order(sales_order_id) ON DELETE CASCADE,
8   FOREIGN KEY (medicine_id) REFERENCES medicine_stock(medicine_id)
9 );
```

- A Composite Primary Key was formed by combining sales_order_id and medicine_id.
- FOREIGN KEYs were established to link this table to both the sales_order and medicine_stock tables.
- The ON DELETE CASCADE constraint was applied to the sales_order_id foreign key. This meant that if an entire sales order was deleted, all individual line items belonging to that order were also purged from the database automatically.

Data Insertion

After the tables were created, initial data was inserted into the database for testing purposes.

- Client Records: Sample records for two hospitals ('Nawaloka' and 'Asiri') and one pharmacy ('City Pharmacy') were inserted into the client table.

```
Run SQL query/queries on database green_pharmaceuticals:

1 INSERT INTO client (username, password, email, phone_no, building_no, street, city, client_type) VALUES
2 ('nawaloka_admin', 'pass123', 'info@nawaloka.com', '0112345678', 'No. 23', 'Sri Jayewardenepura Mawatha', 'Colombo', 'Hospital'),
3 ('city_pharmacy', 'city789', 'city@pharmacy.lk', '0119876543', '15-A', 'Dalada Veediya', 'Kandy', 'Pharmacy'),
4 ('asiri_central', 'asiri456', 'admin@asiri.lk', '0115556667', 'No. 114', 'Norris Canal Road', 'Colombo', 'Hospital');
```

phpMyAdmin

Server: 127.0.0.1 » Database: green_pharmaceuticals » Table: client

Showing rows 0 - 2 (3 total, Query took 0.0003 seconds.)

SELECT * FROM `client`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	client_id	username	password	email	phone_no	building_no	street	city	client_type
☐ Edit ☐ Copy ☐ Delete	1	nawaloka_admin	pass123	info@nawaloka.com	0112345678	No. 23	Sri Jayewardenepura Mawatha	Colombo	Hospital
☐ Edit ☐ Copy ☐ Delete	2	city_pharmacy	city789	city@pharmacy.lk	0119876543	15-A	Dalada Veediya	Kandy	Pharmacy
☐ Edit ☐ Copy ☐ Delete	3	asiri_central	asiri456	admin@asiri.lk	0115556667	No. 114	Norris Canal Road	Colombo	Hospital

- Sub-type Records: The specific details for these clients were added to the hospital and pharmacy tables using the corresponding client_id values generated by the system.

```
Run SQL query/queries on database green_pharmaceuticals:

1 INSERT INTO hospital (client_id, hospital_name, branch, director_name, ward_count) VALUES
2 (1, 'Nawaloka Hospital', 'Colombo 02', 'Dr. Jayantha Dharmadasa', 15),
3 (3, 'Asiri Central', 'Norris Canal Rd', 'Dr. Manjula Karunaratne', 20);
```

green_pharmaceuticals

- New
- client
- hospital
- medicine_stock
- pharmacy
- sales_item
- sales_order

Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

client_id	hospital_name	branch	director_name	ward_count
1	Nawaloka Hospital	Colombo 02	Dr. Jayantha Dharmadasa	15
3	Asiri Central	Norris Canal Rd	Dr. Manjula Karunaratne	20

Run SQL query/queries on database green_pharmaceuticals:

```
1 INSERT INTO pharmacy (client_id, pharmacy_name, pharmacist_reg_no) VALUES
2 (2, 'City Pharmacy Kandy', 'PH-REG-99081');
```

green_pharmaceuticals

- New
- client
- hospital
- medicine_stock
- pharmacy
- sales_item

Number of rows: 25 Filter rows: Search this table

Extra options

client_id	pharmacy_name	pharmacist_reg_no
2	City Pharmacy Kandy	PH-REG-99081

- **Medicine Inventory:** Common medicines like Paracetamol, Amoxicillin, Metformin, and Cetirizine were added to the medicine_stock table with their respective prices, quantities, and expiry dates.

Run SQL query/queries on database green_pharmaceuticals:

```
1 INSERT INTO medicine_stock (generic_name, brand_name, dosage, unit_price, expiry_date, current_quantity) VALUES
2 ('Paracetamol', 'Panadol', '500mg', 5.50, '2026-12-31', 5000),
3 ('Amoxicillin', 'Amoxil', '250mg', 25.00, '2025-06-15', 1200),
4 ('Metformin', 'Glucophage', '500mg', 15.00, '2027-01-20', 800),
5 ('Cetirizine', 'Zyrtec', '10mg', 12.00, '2025-09-10', 50);
6
```

green_pharmaceuticals

- New
- client
- hospital
- medicine_stock
- pharmacy
- sales_item
- sales_order
- information_schema
- mysql
- performance_schema

Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

medicine_id	generic_name	brand_name	dosage	unit_price	expiry_date	current_quantity
1	Paracetamol	Panadol	500mg	5.50	2026-12-31	5000
2	Amoxicillin	Amoxil	250mg	25.00	2025-06-15	1200
3	Metformin	Glucophage	500mg	15.00	2027-01-20	800
4	Cetirizine	Zyrtec	10mg	12.00	2025-09-10	50

- **Sales Transactions:** A test order was created in the sales_order table for 'Nawaloka Hospital'. Subsequently, individual items for that order were inserted into the sales_item table to verify the relationship between orders and medicines.

Run SQL query/queries on database green_pharmaceuticals:

```
1 INSERT INTO sales_order (order_date, total_amount, status, client_id) VALUES
2 (NOW(), 1100.00, 'Pending', 1);
```

green_pharmaceuticals

- New
- client
- hospital
- medicine_stock
- pharmacy
- sales_item
- sales_order

Number of rows: 25 Filter rows: Search this table

Extra options

sales_order_id	order_date	total_amount	status	client_id
1	2026-01-29 18:29:35	1100.00	Pending	1

Run SQL query/queries on database green_pharmaceuticals:

```
1 INSERT INTO sales_item (sales_order_id, medicine_id, quantity, unit_price) VALUES
2 (1, 1, 100, 5.50),
3 (1, 2, 20, 25.00);
```

green_pharmaceuticals

- New
- client
- hospital
- medicine_stock
- pharmacy
- sales_item
- sales_order

Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

sales_order_id	medicine_id	quantity	unit_price
1	1	100	5.50
1	2	20	25.00

3.2 Implementation of web application

The development of the Greencare Pharmaceutical Management System involved a full-stack approach, integrating modern web technologies to ensure a robust, scalable, and user-friendly experience. The system was divided into three main layers: the Presentation Layer (Frontend), the Business Logic Layer (Backend), and the Data Storage Layer (Database).

System Features and Backend Functions

A. Database Connection Implementation

Feature Description - A centralized script acting as a bridge between the PHP backend and the MySQL database. It promotes code reusability and allows for easy configuration changes.

Backend Implementation - Uses the `mysqli_connect()` function with four parameters: server, username, password, and database name. It includes a fail-safe mechanism to stop execution if the connection is unsuccessful.

Green care pharmaceuticals

Leading the way in Medical imports and distribution.

Welcome, Nawaloka Hospital
Client Control Panel

Organization: **Nawaloka Hospital**
Email: **info@nawaloka.com**

Username: **nawaloka_admin**
Phone No: **0112345678**

Client Type: **Hospital**
City: **Colombo**

LOG
OUT

Online Ordering Platform

Place New Order



Select medicines, specify quantities, and place your order quickly with real-time stock updates.

PLACE ORDER

Order History



Check all your past orders, view invoices, and monitor delivery status in one place.

VIEW PREVIOUS ORDERS

Account Settings



Manage your account details, change passwords, and keep your contact information up to date.

UPDATE YOUR ACCOUNT

Backend Implementation - Managed by handleLogin(). It uses mysqli_real_escape_string for security and identifies user roles (Admin/Client). Organization names are dynamically fetched based on the account type.

```
// 1. handleLogin
function handleLogin($conn, $data) {
    $username = mysqli_real_escape_string($conn, $data["username"]);
    $password = mysqli_real_escape_string($conn, $data["password"]);

    if ($username === "admin" && $password === "admin123") {
        return ["success" => true, "isAdmin" => true, "message" => "Welcome Admin!"];
    }

    $query = "SELECT * FROM client WHERE username = '$username' AND password = '$password'";
    $result = mysqli_query($conn, $query);
    if (mysqli_num_rows($result) > 0) {
        $user = mysqli_fetch_assoc($result);
        $client_id = $user['client_id'];
        $user['organization_name'] = $user['username'];
        if ($user['client_type'] === 'Hospital') {
            $h_res = mysqli_query($conn, "SELECT hospital_name FROM hospital WHERE client_id = '$client_id'");
            if ($row = mysqli_fetch_assoc($h_res)) $user['organization_name'] = $row['hospital_name'];
        } else {
            $p_res = mysqli_query($conn, "SELECT pharmacy_name FROM pharmacy WHERE client_id = '$client_id'");
            if ($row = mysqli_fetch_assoc($p_res)) $user['organization_name'] = $row['pharmacy_name'];
        }
        return ["success" => true, "isAdmin" => false, "user" => $user, "message" => "Login successful!"];
    }
    return ["success" => false, "message" => "Invalid credentials."];
}
```

Query Explanation -

SELECT : Used with a WHERE clause to verify credentials in the client table.

Role-Based SELECT : Retrieves names from hospital or pharmacy tables using client_id.

C. Client-Side Feature: Register

Feature Description - A module for new hospitals and pharmacies to register in the system. It collects credentials and contact details to create a formal account before ordering.

The form is titled "Client Registration". It contains input fields for Username, Password, Email Address, and Phone Number. Below these is a section for "Address Details" with fields for Building No, Street, and City. A "Select Client Type" dropdown menu is set to "Hospital". Below this is a dashed green box labeled "Hospital Details" containing fields for Hospital Name, Branch Name, Director Name, and Ward Count. At the bottom is a green "Register Now" button and a link "Already have an account? Login here".

This is a duplicate of the form in the left image, showing the same "Client Registration" form with fields for general information, address, client type (Hospital), and hospital-specific details, followed by a "Register Now" button and a login link.

Backend Implementation - Handled by registerClient(). It first saves general data to the client table, captures the new client_id, and then inserts specific details into either the hospital or pharmacy table based on the user type.

```
// 9. registerClient
function registerClient($conn, $data) {
    $sql1 = "INSERT INTO client (username, password, email, phone_no, building_no, street, city, client_type)
    VALUES ('{$data['username']}', '{$data['password']}', '{$data['email']}', '{$data['phone_no']}',
    '{$data['building_no']}', '{$data['street']}', '{$data['city']}', '{$data['client_type']}')";

    if (mysqli_query($conn, $sql1)) {
        $id = mysqli_insert_id($conn);
        if ($data['client_type'] === "Hospital") {
            $sql2 = "INSERT INTO hospital (client_id, hospital_name, branch, director_name, ward_count)
            VALUES ('$id', '{$data['hospital_name']}', '{$data['branch']}', '{$data['director_name']}', '{$data['ward_count']}')";
        } else {
            $sql2 = "INSERT INTO pharmacy (client_id, pharmacy_name, pharmacist_reg_no)
            VALUES ('$id', '{$data['pharmacy_name']}', '{$data['pharmacist_reg_no']}')";
        }
        return mysqli_query($conn, $sql2) ? ["success" => true, "message" => "Registration Successful!"] : ["success" => false, "message" => mysqli_error($conn)];
    }
    return ["success" => false, "message" => mysqli_error($conn)];
}
```

Query Explanation-

INSERT (Parent): Saves core data like username and email into the client table.

ID Retrieval: Uses mysqli_insert_id() to get the link for sub-tables.

INSERT (Child): Saves specialized data into hospital or pharmacy tables using the retrieved

ID.

Consistency: Maintains a strict 1:1 relationship between account and organization.

D. Client-Side Feature: Place Orders

Feature Description.. A two-part system allowing clients to browse real-time inventory and submit digital orders. It bridges the gap between available stock and the final transaction.

— Back to Dashboard

Available Medicines

ID	Brand	Price	Stock	Qty	Add
1	Panadol	Rs. 5.50	4799	100	Add
2	Amoxil	Rs. 25.00	1200	500	Add
3	Glucophage	Rs. 15.00	800	1	Add
5	Cee-Vit	Rs. 4.00	1	1	Add
12	Lipitor	Rs. 500.00	4999	1	Add

Your Order Bill

Medicine ID	Unit Price (Rs.)	Quantity	Total (Rs.)	
1	Rs. 5.5	100	Rs. 550.00	Remove
2	Rs. 25	500	Rs. 12500.00	Remove

Total: Rs. 13050.00

Submit Order Clear Bill

Part 1: Fetching Available Stocks (clientGetMedicine)

Backend - Retrieves only valid medicines from medicine_stock that are in stock and not expired.

```
// 1. client_get_medicine
function clientGetMedicine($conn) {
    $query = "SELECT * FROM medicine_stock WHERE current_quantity > 0 AND expiry_date > CURDATE()";
    $result = mysqli_query($conn, $query);
    $list = [];
    while ($row = mysqli_fetch_assoc($result)) { $list[] = $row; }
    return $list;
}
```

Queries - *SELECT Filters items where current_quantity > 0.

Validation - Uses expiry_date > CURDATE() to hide expired products for safety.

Part 2: Order Submission (submitOrder)

```
// 2. submitOrder
function submitOrder($conn, $data) {
    $items = $data['items'];
    $total = $data['total'];
    $client_id = $data['client_id'];

    $sql = "INSERT INTO sales_order (total_amount, status, client_id) VALUES ('$total', 'Pending', '$client_id')";

    if (mysqli_query($conn, $sql)) {
        $orderId = mysqli_insert_id($conn);
        foreach ($items as $item) {
            $m_id = $item['medicine_id'];
            $qty = $item['quantity'];
            $price = $item['unit_price'];
            mysqli_query($conn, "INSERT INTO sales_item (sales_order_id, medicine_id, quantity, unit_price) VALUES ('$orderId', '$m_id', '$qty', '$price')");
        }
        return ["success" => true, "message" => "Order Success! "];
    }
    return ["success" => false, "message" => mysqli_error($conn)];
}
```

Backend - Records the main order first, then uses a loop to save each individual item selected by the client.

Queries -

INSERT (Main): Creates a record in sales_order with the total amount and 'Pending' status.

Linking: Uses mysqli_insert_id() to connect items to the main order.

INSERT (Items): Adds multiple records to sales_item linked via the Order ID.

E. Client-Side Feature: View Previous Orders

Feature Description - An order tracking module allowing clients to monitor purchase history and order statuses (e.g., Pending, Approved) for better transparency and record-keeping.

My Previous Orders

ORDER ID	TOTAL AMOUNT (RS.)	STATUS	ORDER DATE
1	Rs. 1100.00	Cancelled	2026-01-29 18:29:35
4	Rs. 754.00	Cancelled	2026-02-01 10:11:01
9	Rs. 500.00	Approved	2026-02-01 20:56:53
10	Rs. 19840.00	Approved	2026-02-01 20:59:46
11	Rs. 8000.00	Approved	2026-02-01 21:10:38
12	Rs. 500.00	Pending	2026-02-01 23:13:24
13	Rs. 500.00	Pending	2026-02-01 23:13:28
14	Rs. 500.00	Pending	2026-02-01 23:15:08
15	Rs. 13050.00	Pending	2026-02-02 11:17:36

[← Back to Dashboard](#)

Part 1: Order History (getClientPreviousOrders)

Backend - Fetches summary data (date, total, status) for a specific client while ensuring security through data sanitization.

```
// 2. get_client_previous_orders
function getClientPreviousOrders($conn, $c_id) {
    $c_id = mysqli_real_escape_string($conn, $c_id);
    $query = "SELECT * FROM sales_order WHERE client_id = '$c_id'";
    $result = mysqli_query($conn, $query);
    $orders = [];
    while ($row = mysqli_fetch_assoc($result)) { $orders[] = $row; }
    return ["success" => true, "data" => $orders];
}
```

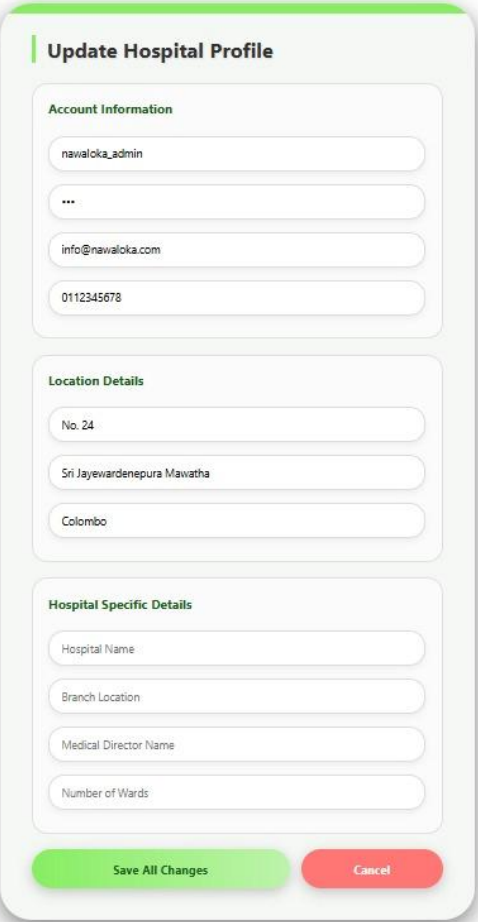
Queries -

*SELECT : Fetches data from sales_order

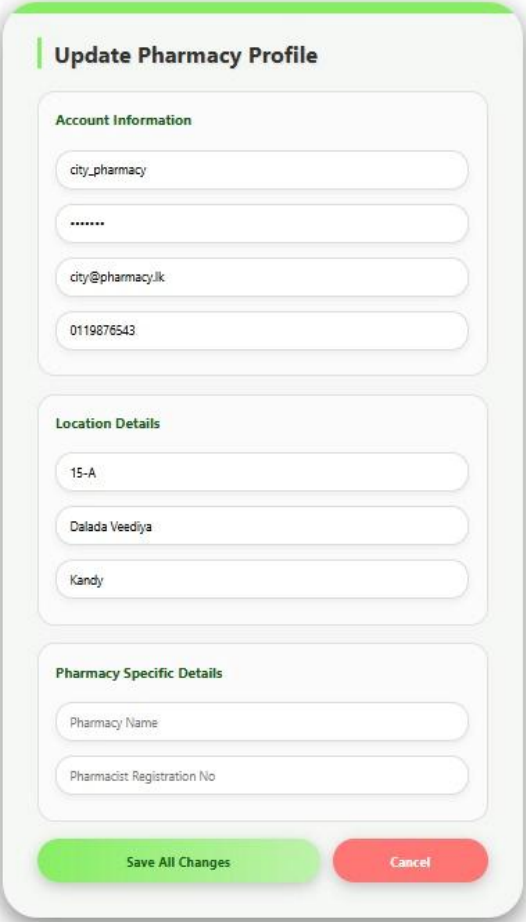
Filtering - Uses a WHERE clause with client_id to restrict access to the user's own records only.

F. Client-Side Feature: Update Account

Feature Description - A profile management module allowing clients to update contact and organizational details, ensuring the distributor maintains accurate communication channels.



The 'Update Hospital Profile' form is divided into three main sections: Account Information, Location Details, and Hospital Specific Details. The Account Information section contains four input fields with pre-filled values: 'newaloka_admin', '...', 'info@newaloka.com', and '0112345678'. The Location Details section contains three input fields with pre-filled values: 'No. 24', 'Sri Jayewardenepura Mawatha', and 'Colombo'. The Hospital Specific Details section contains four input fields with pre-filled values: 'Hospital Name', 'Branch Location', 'Medical Director Name', and 'Number of Wards'. At the bottom, there are two buttons: 'Save All Changes' (green) and 'Cancel' (red).



The 'Update Pharmacy Profile' form is divided into three main sections: Account Information, Location Details, and Pharmacy Specific Details. The Account Information section contains four input fields with pre-filled values: 'city_pharmacy', '*****', 'city@pharmacy.lk', and '0119876543'. The Location Details section contains three input fields with pre-filled values: '15-A', 'Dalada Veediya', and 'Kandy'. The Pharmacy Specific Details section contains two input fields with pre-filled values: 'Pharmacy Name' and 'Pharmacist Registration No'. At the bottom, there are two buttons: 'Save All Changes' (green) and 'Cancel' (red).

Part 1: Updating Hospital Details (updateHospitalProfile)

Backend - Synchronizes changes across the client and hospital tables using the unique

client_id.

```
// 7. updateHospitalProfile
function updateHospitalProfile($conn, $data) {
    $c_id = $data['client_id'];
    $sql1 = "UPDATE client SET username='{$data['username']}' , email='{$data['email']}' , phone_no='{$data['phone_no']}' , city='{$data['city']}' WHERE client_id='{$c_id}'";
    $sql2 = "UPDATE hospital SET hospital_name='{$data['hospital_name']}' , branch='{$data['branch']}' WHERE client_id='{$c_id}'";
    return (mysqli_query($conn, $sql1) && mysqli_query($conn, $sql2)) ? ["success" => true] : ["success" => false];
}
```

Queries –

*UPDATE (Account): Modifies common fields like username and email in the client table.

UPDATE (Specifics): Updates institutional fields like hospital_name and branch.

Targeting: Uses a WHERE clause to ensure only the logged-in client's record is altered.

Part 2: Updating Pharmacy Details (updatePharmacyProfile)

Backend - Updates general account data and specific pharmacy credentials (e.g., pharmacist registration number) simultaneously.

```
// 8. updatePharmacyProfile
function updatePharmacyProfile($conn, $data) {
    $c_id = $data['client_id'];
    $sql1 = "UPDATE client SET username='{$data['username']}' , email='{$data['email']}' , phone_no='{$data['phone_no']}' WHERE client_id='{$c_id}'";
    $sql2 = "UPDATE pharmacy SET pharmacy_name='{$data['pharmacy_name']}' , pharmacist_reg_no='{$data['pharmacist_reg_no']}' WHERE client_id='{$c_id}'";
    return (mysqli_query($conn, $sql1) && mysqli_query($conn, $sql2)) ? ["success" => true] : ["success" => false];
}
```

Queries -

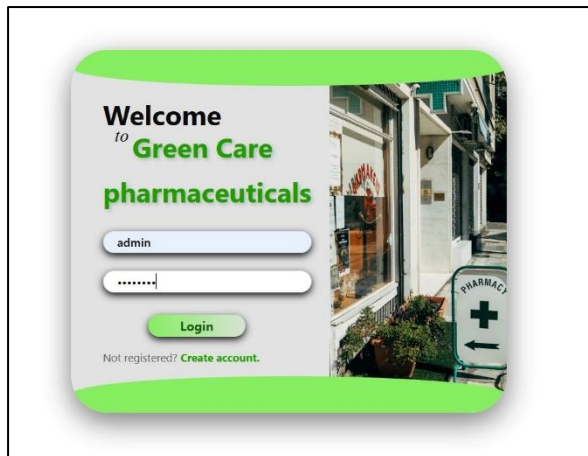
Synchronized UPDATE: Triggers two operations to maintain consistency between the user account and the pharmacy profile.

Specificity: Updates unique pharmacy fields like pharmacist_reg_no.

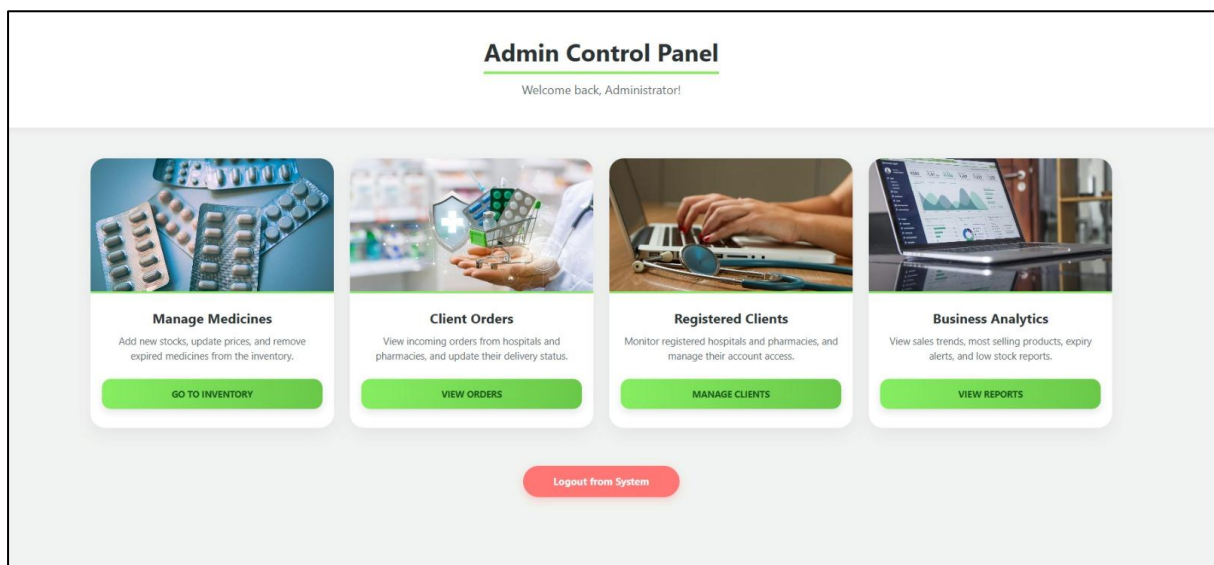
Integrity: Uses client_id as the primary identifier to prevent accidental changes to other accounts.

G. Admin-Side Feature: Login

Feature Description - A restricted portal for administrators to access the Control Panel. It distinguishes between admin and regular client roles to ensure high-level security.



```
// admin username & password (Hard-coded)
if ($username === "admin" && $password === "admin123") {
    echo json_encode([
        "success" => true,
        "isAdmin" => true,
        "message" => "Welcome to Admin Panel!"
    ]);
}
```



Backend Implementation - Integrated into the `handleLogin()` function. It uses a dual-layer approach: a hard-coded check validates the administrator (credentials: admin/admin123), while a database query verifies regular clients.

```
// 1. handleLogin
function handleLogin($conn, $data) {
    $username = mysqli_real_escape_string($conn, $data["username"]);
    $password = mysqli_real_escape_string($conn, $data["password"]);

    if ($username === "admin" && $password === "admin123") {
        return ["success" => true, "isAdmin" => true, "message" => "Welcome Admin!"];
    }

    $query = "SELECT * FROM client WHERE username = '$username' AND password = '$password'";
    $result = mysqli_query($conn, $query);
    if (mysqli_num_rows($result) > 0) {
        $user = mysqli_fetch_assoc($result);
        $client_id = $user['client_id'];
        $user['organization_name'] = $user['username'];
        if ($user['client_type'] === 'Hospital') {
            $h_res = mysqli_query($conn, "SELECT hospital_name FROM hospital WHERE client_id = '$client_id'");
            if ($row = mysqli_fetch_assoc($h_res)) $user['organization_name'] = $row['hospital_name'];
        } else {
            $p_res = mysqli_query($conn, "SELECT pharmacy_name FROM pharmacy WHERE client_id = '$client_id'");
            if ($row = mysqli_fetch_assoc($p_res)) $user['organization_name'] = $row['pharmacy_name'];
        }
        return ["success" => true, "isAdmin" => false, "user" => $user, "message" => "Login successful!"];
    }
    return ["success" => false, "message" => "Invalid credentials."];
}
```

Query Explanation -

Logic-Based Check: Uses an IF condition for admin credentials to bypass database lookups for faster, secure access.

SELECT: If the admin check fails, the system queries the client table to validate the user.

WHERE Clause: Filters by username and password to ensure only valid users are authenticated.

Sanitization: Uses `mysqli_real_escape_string` to protect the admin portal from SQL injection.

H. Admin-Side Feature: Manage Medicines

Feature Description - A comprehensive inventory control module for the administrator to add, view, and remove medicines, ensuring the digital catalog stays synchronized with physical stock.

[-- Back to Dashboard](#)

Add New Medicine

Asprin	Hylos	300mg	12.95	03/24/2026	
5000	<button>Add Medicine</button>				

Available medicines in Medicine stock

⚠ Notice for Inventory Management:

Medicines linked to past or pending orders cannot be deleted. Removing these items would erase critical **product history and financial information** from your sales reports. To manage inventory for these items, we recommend updating the stock levels or pricing instead of deletion.

ID	Brand (Generic)	Dosage	Price (Rs.)	Stock Level	Expiry Date	Action
1	Panadol (Paracetamol)	500mg	5.50	4799	2026-12-31	<button>Delete</button>
2	Amoxil (Amoxicillin)	250mg	25.00	1200	2025-06-15	<button>Delete</button>
3	Glucophage (Metformin)	500mg	15.00	800	2027-01-20	<button>Delete</button>
5	Cee-Vit (Vitamin C)	500mg	4.00	1	2028-10-10	<button>Delete</button>
12	Lipitor (Atorvastatin)	10mg	500.00	4999	2026-03-31	<button>Delete</button>

Available medicines in Medicine stock

⚠ Notice for Inventory Management:

Medicines linked to past or pending orders cannot be deleted. Removing these items would erase critical **product history and financial information** from your sales reports. To manage inventory for these items, we recommend updating the stock levels or pricing instead of deletion.

ID	Brand (Generic)	Dosage	Price (Rs.)	Stock Level	Expiry Date	Action
1	Panadol (Paracetamol)	500mg	5.50	4799	2026-12-31	<button>Delete</button>
2	Amoxil (Amoxicillin)	250mg	25.00	1200	2025-06-15	<button>Delete</button>
3	Glucophage (Metformin)	500mg	15.00	800	2027-01-20	<button>Delete</button>
5	Cee-Vit (Vitamin C)	500mg	4.00	1	2028-10-10	<button>Delete</button>
12	Lipitor (Atorvastatin)	10mg	500.00	4999	2026-03-31	<button>Delete</button>
14	Hylos (Asprin)	300mg	12.95	5000	2026-03-24	<button>Delete</button>

Part 1: Adding Inventory (addMedicine)

Backend - Captures and sanitizes product details (names, dosage, price) before inserting them into the inventory table.

```
// 4. addMedicine
function addMedicine($conn, $data) {
    $g = mysqli_real_escape_string($conn, $data['generic_name'] ?? '');
    $b = mysqli_real_escape_string($conn, $data['brand_name'] ?? '');
    $d = mysqli_real_escape_string($conn, $data['dosage'] ?? '');
    $p = mysqli_real_escape_string($conn, $data['unit_price'] ?? 0);
    $e = mysqli_real_escape_string($conn, $data['expiry_date'] ?? '');
    $q = mysqli_real_escape_string($conn, $data['current_quantity'] ?? 0);

    $sql = "INSERT INTO medicine_stock (generic_name, brand_name, dosage, unit_price, expiry_date, current_quantity)
            VALUES ('$g', '$b', '$d', '$p', '$e', '$q')";

    if (mysqli_query($conn, $sql)) {
        return ["success" => true, "message" => "Medicine added successfully!"];
    } else {
        return ["success" => false, "message" => "Database Error: " . mysqli_error($conn)];
    }
}
```

Queries - Uses INSERT INTO to map sanitized product attributes to the medicine_stock table.

Part 2: Safe Deletion (deleteMedicine)

Backend - Implements a safety check to prevent deleting medicines that have a transaction history, preserving financial records.

```
// 5. deleteMedicine
function deleteMedicine($conn, $data) {
    $id = mysqli_real_escape_string($conn, $data['medicine_id'] ?? '');

    $checkQuery = "SELECT COUNT(*) as item_count FROM sales_item WHERE medicine_id = '$id'";
    $result = mysqli_query($conn, $checkQuery);
    $row = mysqli_fetch_assoc($result);

    if ($row['item_count'] > 0) {
        return [
            "success" => false,
            "message" => "Cannot delete! This medicine is linked to previous orders and must be kept for sales history. (Exclamation!)"
        ];
    }

    $sql = "DELETE FROM medicine_stock WHERE medicine_id = '$id'";
    if (mysqli_query($conn, $sql)) {
        return ["success" => true, "message" => "Medicine deleted successfully!"];
    } else {
        return ["success" => false, "message" => "Database error occurred."];
    }
}
```

Queries –

* Verification: Runs SELECT COUNT(*) on sales_item to check for existing links.

DELETE: Only executes if the count is zero, maintaining relational integrity.

Part 3: Inventory Retrieval (getAllMedicines)

Backend - Fetches the entire catalog to help the admin monitor stock levels, pricing, and expiry dates.

```
// 5. get_all_medicines
function getAllMedicines($conn) {
    $result = mysqli_query($conn, "SELECT * FROM medicine_stock");
    $medicines = [];
    while($row = mysqli_fetch_assoc($result)) { $medicines[] = $row; }
    return ["success" => true, "data" => $medicines];
}
```

Queries - Uses a ****SELECT **** query on medicine_stock and stores results in an array for dashboard display.

I. Admin-Side Feature: Client Orders Management

Feature Description - A central module for the administrator to process orders. It includes a critical synchronization feature that automatically updates inventory levels based on whether an order is approved or canceled.

[← Back to Dashboard](#)

Admin Order Management

Select Order Status and click to Update

Order ID	Client Name	Amount (Rs.)	Date	Status	Action	View Items
#2	city_pharmacy (undefined)	1850.00	2026-01-31 11:50:02	Approved	<button>Update</button>	<button>View Items</button>
#5	city_pharmacy (undefined)	19.00	2026-02-01 10:26:46	Cancelled	<button>Update</button>	<button>View Items</button>
#6	city_pharmacy (undefined)	5.50	2026-02-01 10:38:24	Cancelled	<button>Update</button>	<button>View Items</button>
#3	nawalapitiya_health (undefined)	237.50	2026-01-31 19:09:03	Approved	<button>Update</button>	<button>View Items</button>
#1	nawaloka_admin (undefined)	1100.00	2026-01-29 18:29:35	Cancelled	<button>Update</button>	<button>View Items</button>
#4	nawaloka_admin (undefined)	754.00	2026-02-01 10:11:01	Cancelled	<button>Update</button>	<button>View Items</button>
#9	nawaloka_admin (undefined)	500.00	2026-02-01 20:56:53	Approved	<button>Update</button>	<button>View Items</button>
#10	nawaloka_admin (undefined)	19840.00	2026-02-01 20:59:46	Approved	<button>Update</button>	<button>View Items</button>
#11	nawaloka_admin (undefined)	8000.00	2026-02-01 21:10:38	Approved	<button>Update</button>	<button>View Items</button>
#12	nawaloka_admin (undefined)	500.00	2026-02-01 23:13:24	Pending	<button>Update</button>	<button>View Items</button>
#13	nawaloka_admin (undefined)	500.00	2026-02-01 23:13:28	Pending	<button>Update</button>	<button>View Items</button>
#14	nawaloka_admin (undefined)	500.00	2026-02-01 23:15:08	Pending	<button>Update</button>	<button>View Items</button>
#15	nawaloka_admin (undefined)	13050.00	2026-02-02 11:17:36	Pending	<button>Update</button>	<button>View Items</button>

#13	nawaloka_admin (undefined)	500.00	2026-02-01 23:13:28	Pending	<button>Update</button>	<button>View Items</button>
#14	nawaloka_admin (undefined)	500.00	2026-02-01 23:15:08	Approved	<button>Update</button>	<button>View Items</button>
#15	nawaloka_admin (undefined)	13050.00	2026-02-02 11:17:36	Pending	<button>Update</button>	<button>View Items</button>

Admin Order Management

Select Order Status and click to Update

Order ID	Client Name	Amount (Rs.)	Date	Status	Action	View Items
#2	city_pharmacy (undefined)	1850.00	2026-01-31 11:50:02	Approved	<button>Update</button>	<button>View Items</button>
#5	city_pharmacy (undefined)	19.00	2026-02-01 10:26:46	Cancelled	<button>Update</button>	<button>View Items</button>
#6	city_pharmacy (undefined)	5.50	2026-02-01 10:38:24	Cancelled	<button>Update</button>	<button>View Items</button>
#3	nawalapitiya_health (undefined)	237.50	2026-01-31 19:09:03	Approved	<button>Update</button>	<button>View Items</button>
#1	nawaloka_admin (undefined)	1100.00	2026-01-29 18:29:35	Cancelled	<button>Update</button>	<button>View Items</button>
#4	nawaloka_admin (undefined)	754.00	2026-02-01 10:11:01	Cancelled	<button>Update</button>	<button>View Items</button>
#9	nawaloka_admin (undefined)	500.00	2026-02-01 20:56:53	Approved	<button>Update</button>	<button>View Items</button>
#10	nawaloka_admin (undefined)	19840.00	2026-02-01 20:59:46	Approved	<button>Update</button>	<button>View Items</button>
#11	nawaloka_admin (undefined)	8000.00	2026-02-01 21:10:38	Approved	<button>Update</button>	<button>View Items</button>
#12	nawaloka_admin (undefined)	500.00	2026-02-01 23:13:24	Pending	<button>Update</button>	<button>View Items</button>
#13	nawaloka_admin (undefined)	500.00	2026-02-01 23:13:28	Pending	<button>Update</button>	<button>View Items</button>
#14	nawaloka_admin (undefined)	500.00	2026-02-01 23:15:08	Pending	<button>Update</button>	<button>View Items</button>
#15	nawaloka_admin (undefined)	13050.00	2026-02-02 11:17:36	Pending	<button>Update</button>	<button>View Items</button>

Order Details - ID: 15

Medicine Name	Qty	Unit Price	Subtotal
Panadol Paracetamol	100	5.50	550.00
Amoxil Amoxicillin	500	25.00	12500.00

Part 1: Monitoring Orders (getAllOrders)

Backend - Retrieves all transactions while linking them to client identities for a more informative overview.

```
// 3. get_all_orders
function getAllOrders($conn) {
    $sql = "SELECT sales_order.*, client.username
            FROM sales_order
            JOIN client ON sales_order.client_id = client.client_id";
    $result = mysqli_query($conn, $sql);
    $orders = [];
    while($row = mysqli_fetch_assoc($result)) { $orders[] = $row; }
    return ["success" => true, "data" => $orders];
}
```

Queries - Uses a JOIN between sales_order and client to fetch the client's username alongside order details.

Part 2: Status & Inventory Sync (updateOrderStatus)

Backend - Performs logic-based stock adjustments. It deducts stock on 'Approval' and restores it on 'Cancellation'.

```
// 3. updateOrderStatus
function updateOrderStatus($conn, $data) {
    $id = mysqli_real_escape_string($conn, $data['order_id']);
    $status = mysqli_real_escape_string($conn, $data['status']);

    $checkStatus = mysqli_query($conn, "SELECT status FROM sales_order WHERE sales_order_id = '$id'");
    $row = mysqli_fetch_assoc($checkStatus);
    $currentStatus = strtolower($row['status']);
    $newStatus = strtolower($status);

    if ($currentStatus === 'pending' && $newStatus === 'approved') {
        $itemsQuery = mysqli_query($conn, "SELECT medicine_id, quantity FROM sales_item WHERE sales_order_id = '$id'");
        while ($item = mysqli_fetch_assoc($itemsQuery)) {
            $medId = $item['medicine_id'];
            $qty = $item['quantity'];

            mysqli_query($conn, "UPDATE medicine_stock SET current_quantity = current_quantity - $qty
                                WHERE medicine_id = '$medId' AND current_quantity >= $qty");
        }
    }

    if (($newStatus === 'canceled' || $newStatus === 'cancelled') && ($currentStatus === 'approved' || $currentStatus === 'dispatched')) {
        $itemsQuery = mysqli_query($conn, "SELECT medicine_id, quantity FROM sales_item WHERE sales_order_id = '$id'");
        while ($item = mysqli_fetch_assoc($itemsQuery)) {
            $medId = $item['medicine_id'];
            $qty = $item['quantity'];
            mysqli_query($conn, "UPDATE medicine_stock SET current_quantity = current_quantity + $qty WHERE medicine_id = '$medId'");
        }
    }

    $sql = "UPDATE sales_order SET status = '$status' WHERE sales_order_id = '$id'";
    return mysqli_query($conn, $sql) ? ["success" => true, "message" => "Status updated successfully!"] : ["success" => false];
}
```

Queries -

Verification: SELECT current status first.

Deduction: UPDATE with current_quantity - \$qty (ensuring quantity >= 0).

Restoration: UPDATE with current_quantity + \$qty.

Status: Final UPDATE to change the order status.

Part 3: Order Contents (getOrderItems)

Backend - Reuses client-side logic to show brand and generic names for medicines within a specific order.

```
// 4. get_order_items
function getOrderItems($conn, $order_id) {
    $order_id = mysqli_real_escape_string($conn, $order_id);
    $sql = "SELECT si.*, m.brand_name, m.generic_name
            FROM sales_item si
            JOIN medicine_stock m ON si.medicine_id = m.medicine_id
            WHERE si.sales_order_id = '$order_id'";
    $result = mysqli_query($conn, $sql);
    $items = [];
    while($row = mysqli_fetch_assoc($result)) { $items[] = $row; }
    return ["success" => true, "data" => $items];
}
```

Queries - Uses an INNER JOIN on sales_item and medicine_stock filtered by the sales_order_id. (Please insert screenshot here)

J. Admin-Side Feature: Registered Clients Management

Feature Description - A unified interface for managing hospitals and pharmacies. It allows administrators to review profiles and safely remove inactive accounts while maintaining business record integrity.

[← Back to Dashboard](#)

Registered Clients

⚠ Important Note on Deletion:

You cannot delete a client who has existing orders in the system. This is to ensure that **historical business data and sales records** are preserved for future analytics and audit purposes. If you must remove this account, please ensure all linked orders are processed or archived first.

ID	USERNAME	ORGANIZATION DETAILS	CLIENT TYPE	ADDRESS	CONTACT INFO	ACTIONS
1	nawaloka_admin	Nawaloka Hospital Branch: Colombo 02 Director: Dr. Jayantha Dharmadasa Wards: 20	Hospital	Colombo Sri Jayewardenepura Mawatha, No. 24	0112345678 info@nawaloka.com	Delete
2	city_pharmacy	City Pharmacy Kandy Reg No: PH-REG-99081	Pharmacy	Kandy Dalada Veediya, 15-A	0119876543 city@pharmacy.lk	Delete
7	nawalapitiya_health	Nawalapitiya Medical Center Branch: Central Branch Director: Dr. S. Bandara Wards: 5	Hospital	Nawalapitiya Station Road, No. 05	0542233445 info@nhealth.com	Delete

Part 1: Comprehensive Client Profiles (getAllClients)

Backend - Aggregates data from client, hospital, and pharmacy tables into a single result set to show a complete profile for each entity.

```
// 6. get_all_clients
function getAllClients($conn) {
    $sql = "SELECT c.*, h.hospital_name, h.branch, h.director_name, h.ward_count,
                p.pharmacy_name, p.pharmacist_reg_no
            FROM client c
            LEFT JOIN hospital h ON c.client_id = h.client_id
            LEFT JOIN pharmacy p ON c.client_id = p.client_id";
    $result = mysqli_query($conn, $sql);
    $clients = [];
    while($row = mysqli_fetch_assoc($result)) { $clients[] = $row; }
    return ["success" => true, "data" => $clients];
}
```

Queries -

* LEFT JOIN: Links the main client table to both sub-tables.

Flexible Mapping: Ensures that specific fields (like ward_count or reg_no) are retrieved regardless of the client type.

Part 2: Secure Client Deletion (deleteClient)

Backend - Follows a strict integrity policy. It blocks deletion if a client has existing orders and uses a sequential process to remove records from sub-tables before the main account.

```
// 6. deleteClient
function deleteClient($conn, $data) {
    $id = mysqli_real_escape_string($conn, $data['client_id']);

    $orderCheckQuery = "SELECT COUNT(*) as order_count FROM sales_order WHERE client_id = '$id'";
    $result = mysqli_query($conn, $orderCheckQuery);
    $row = mysqli_fetch_assoc($result);

    if ($row['order_count'] > 0) {
        return [
            "success" => false,
            "message" => "This client has placed orders and cannot be deleted to preserve business history!"
        ];
    }

    mysqli_query($conn, "DELETE FROM hospital WHERE client_id = '$id'");
    mysqli_query($conn, "DELETE FROM pharmacy WHERE client_id = '$id'");
    $res = mysqli_query($conn, "DELETE FROM client WHERE client_id = '$id'");

    return $res ? ["success" => true, "message" => "Client deleted successfully!"] : ["success" => false];
}
```

Queries -

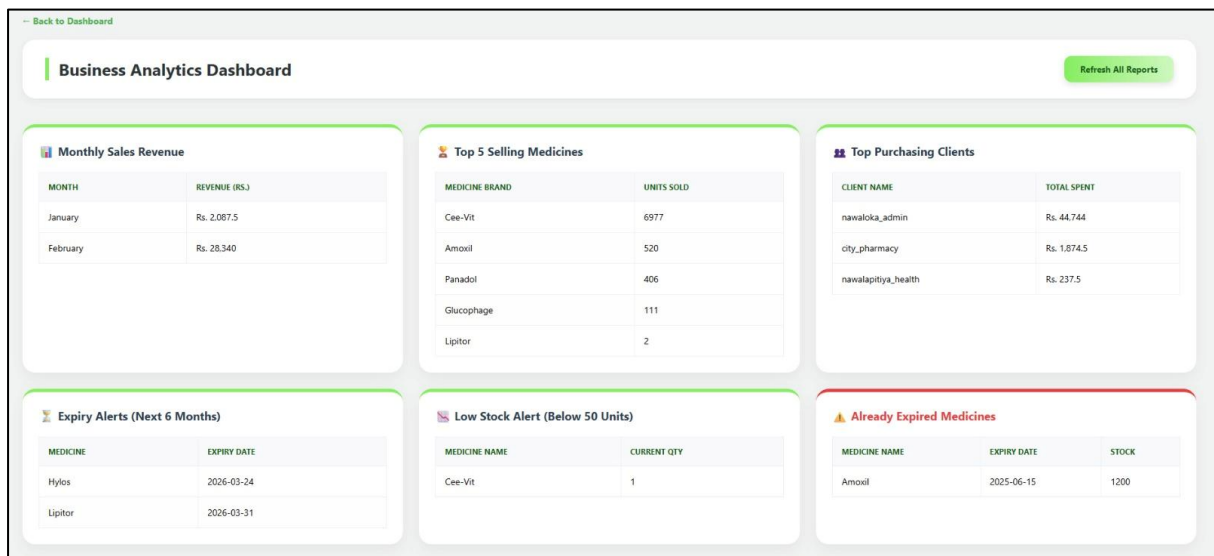
Validation: Runs `SELECT COUNT(*)` on `sales_order` to safeguard transaction history.

Sequential DELETE: Removes records from `hospital` and `pharmacy` child tables first.

Final Removal: Deletes the entry from the `client` table once all dependencies are cleared.

K. Admin-Side Feature: Business Analytics & Reporting

Feature Description - A decision-support module that transforms raw data into actionable insights. It provides reports on sales performance, inventory health, and client trends to help the administrator make informed business decisions.



Part 1: Sales and Performance Metrics

Backend - Uses `getMonthlySales()`, `getMostSellingMedicines()`, and `getTopClients()` to calculate revenue and rankings directly within the database for high performance.

```
/* ----- REPORT FUNCTIONS ----- */

// 1. Monthly Sales Report
function getMonthlySales($conn) {
    $sql = "SELECT MONTHNAME(order_date) as month, SUM(total_amount) as total
            FROM sales_order WHERE status = 'Approved'
            GROUP BY MONTH(order_date)";
    $result = mysqli_query($conn, $sql);
    $data = [];
    while($row = mysqli_fetch_assoc($result)) { $data[] = $row; }
    return ["success" => true, "data" => $data];
}

// 2. Most Selling Medicines
function getMostSellingMedicines($conn) {
    $sql = "SELECT m.brand_name, SUM(si.quantity) as total_qty
            FROM sales_item si
            JOIN medicine_stock m ON si.medicine_id = m.medicine_id
            GROUP BY si.medicine_id ORDER BY total_qty DESC LIMIT 5";
    $result = mysqli_query($conn, $sql);
    $data = [];
    while($row = mysqli_fetch_assoc($result)) { $data[] = $row; }
    return ["success" => true, "data" => $data];
}

// 3. Top Purchasing Clients
function getTopClients($conn) {
    $sql = "SELECT c.username, SUM(so.total_amount) as total_spent
            FROM sales_order so
            JOIN client c ON so.client_id = c.client_id
            GROUP BY so.client_id ORDER BY total_spent DESC LIMIT 5";
    $result = mysqli_query($conn, $sql);
    $data = [];
    while($row = mysqli_fetch_assoc($result)) { $data[] = $row; }
    return ["success" => true, "data" => $data];
}
```

Queries –

* Revenue Grouping: Uses `SUM(total_amount)` and `GROUP BY MONTH()` to track monthly income for approved orders.

Popularity Ranking: Joins `sales_item` and `medicine_stock` to find top 5 products using `SUM(quantity)` and `ORDER BY DESC`.

Customer Value: Groups by `client_id` to identify the highest-spending organizations.

Part 2: Inventory Health and Expiry Management

Backend - Functions as an early warning system to identify low stock, nearing expiry, or already expired medicines.

```
// 4. Expiry Date Report (Next 6 Months)
function getExpiryReport($conn) {
    // CURDATE()
    $sql = "SELECT brand_name, expiry_date FROM medicine_stock
            WHERE expiry_date > CURDATE()
            AND expiry_date <= DATE_ADD(CURDATE(), INTERVAL 6 MONTH)
            ORDER BY expiry_date ASC";

    $result = mysqli_query($conn, $sql);
    $data = [];
    while($row = mysqli_fetch_assoc($result)) { $data[] = $row; }
    return ["success" => true, "data" => $data];
}

// 5. Low Stock Alert
function getLowStockReport($conn) {
    $sql = "SELECT brand_name, current_quantity FROM medicine_stock
            WHERE current_quantity < 50";
    $result = mysqli_query($conn, $sql);
    $data = [];
    while($row = mysqli_fetch_assoc($result)) { $data[] = $row; }
    return ["success" => true, "data" => $data];
}

// 6. Already Expired Medicines Report
function getExpiredMedicines($conn) {
    //
    $sql = "SELECT brand_name, expiry_date, current_quantity
            FROM medicine_stock
            WHERE expiry_date <= CURDATE()
            ORDER BY expiry_date DESC";

    $result = mysqli_query($conn, $sql);
    $data = [];
    if($result) {
        while($row = mysqli_fetch_assoc($result)) {
            $data[] = $row;
        }
    }
    return ["success" => true, "data" => $data];
}
```

Queries -

Expiry Tracking: Uses DATE_ADD(CURDATE(), INTERVAL 6 MONTH) to find items expiring soon.

Stock Alerts: Filters for current_quantity < 50 to trigger reordering.

Loss Identification: Uses expiry_date <= CURDATE() to flag expired stock for disposal.

3.3 Assumptions for System Implementation

The following assumptions were made during the development and implementation of the Greencare Pharmaceutical Management System ,

- **Unique Client Identification:** It was assumed that every hospital or pharmacy has a unique username and email address. This prevents duplicate accounts and ensures data consistency.
- **Fixed Unit Pricing:** The unit price of a medicine is assumed to be fixed at the time the order is placed. Even if the inventory price changes later, the cost of previously placed orders remains the same.
- **Stock Adjustment Policy:** It was assumed that stock levels are only reduced when an order is "Approved" by the Admin. No stock is reserved or deducted while an order is in the "Pending" state.
- **Standardized Units and Currency:** It was assumed that all financial transactions are conducted in a single currency (e.g., LKR) and that all medicine quantities are measured in standard units (e.g., counts or tablets).
- **Data Accuracy by Administrator:** It was assumed that all data entered by the administrator, such as medicine expiry dates and initial stock levels, are accurate and verified.
- **Business History Preservation:** A key assumption was made regarding data deletion. To protect business history, the ON DELETE CASCADE constraint was avoided for primary entities like Clients and Medicines. It is assumed that no record with an existing transaction history should be fully removed from the system.

4 .Phase IV – Discussion

4.1 Functionality of the database

After several operations in the implemented web application this is how the database was structured (in phpMyAdmin),

Client Table

Extra options										
		client_id	username	password	email	phone_no	building_no	street	city	client_type
☐	Edit Copy Delete	1	nawaloka_admin	123	info@nawaloka.com	0112345678	No. 24	Sri Jayewardenepura Mawatha	Colombo	Hospital
☐	Edit Copy Delete	2	city_pharmacy	city789	city@pharmacy.lk	0119876543	15-A	Dalada Veediya	Kandy	Pharmacy
☐	Edit Copy Delete	7	nawalapitiya_health	pass1234	info@nhealth.com	0542233445	No. 05	Station Road	Nawalapitiya	Hospital
Check all With selected: Edit Copy Delete Export										

Hospital Table

Extra options

			client_id	hospital_name	branch	director_name	ward_count	
☐	 Edit	 Copy	 Delete	1	Nawaloka Hospital	Colombo 02	Dr. Jayantha Dharmadasa	20
☐	 Edit	 Copy	 Delete	7	Nawalapitiya Medical Center	Central Branch	Dr. S. Bandara	5
	☐ Check all	With selected:		 Edit	 Copy	 Delete	 Export	

Pharmacy Table

Extra options

	client_id	pharmacy_name	pharmacist_reg_no
☐ Edit Copy Delete	2	City Pharmacy Kandy	PH-REG-99081
Check all With selected: Edit Copy Delete Export			

Medicine Stock Table

Extra options

			medicine_id	generic_name	brand_name	dosage	unit_price	expiry_date	current_quantity
☐	 Edit	 Copy	 Delete	1	Paracetamol	Panadol	500mg	5.50 2026-12-31	4799
☐	 Edit	 Copy	 Delete	2	Amoxicillin	Amoxil	250mg	25.00 2025-06-15	1200
☐	 Edit	 Copy	 Delete	3	Metformin	Glucophage	500mg	15.00 2027-01-20	800
☐	 Edit	 Copy	 Delete	5	Vitamin C	Cee-Vit	500mg	4.00 2028-10-10	1
☐	 Edit	 Copy	 Delete	12	Atorvastatin	Lipitor	10mg	500.00 2026-03-31	4999
☐	 Edit	 Copy	 Delete	14	Aspirin	Hylos	300mg	12.95 2026-03-24	5000
	☐ Check all	With selected:	 Edit	 Copy	 Delete	 Export			

Sales Order Table

☐ Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

				sales_order_id	order_date	total_amount	status	client_id
☐	Edit	Copy	Delete	1	2026-01-29 18:29:35	1100.00	Cancelled	1
☐	Edit	Copy	Delete	2	2026-01-31 11:50:02	1850.00	Approved	2
☐	Edit	Copy	Delete	3	2026-01-31 19:09:03	237.50	Approved	7
☐	Edit	Copy	Delete	4	2026-02-01 10:11:01	754.00	Cancelled	1
☐	Edit	Copy	Delete	5	2026-02-01 10:26:46	19.00	Cancelled	2
☐	Edit	Copy	Delete	6	2026-02-01 10:38:24	5.50	Cancelled	2
☐	Edit	Copy	Delete	9	2026-02-01 20:56:53	500.00	Approved	1
☐	Edit	Copy	Delete	10	2026-02-01 20:59:46	19840.00	Approved	1
☐	Edit	Copy	Delete	11	2026-02-01 21:10:38	8000.00	Approved	1
☐	Edit	Copy	Delete	12	2026-02-01 23:13:24	500.00	Pending	1
☐	Edit	Copy	Delete	13	2026-02-01 23:13:28	500.00	Pending	1
☐	Edit	Copy	Delete	14	2026-02-01 23:15:08	500.00	Pending	1
☐	Edit	Copy	Delete	15	2026-02-02 11:17:36	13050.00	Pending	1

Sales Item Table

Extra options

						sales_order_id	medicine_id	quantity	unit_price			
		Edit		Copy		Delete	1	1	100	5.50		
		Edit		Copy		Delete	1	2	20	25.00		
		Edit		Copy		Delete	2	1	200	5.50		
		Edit		Copy		Delete	2	3	50	15.00		
		Edit		Copy		Delete	3	1	5	5.50		
		Edit		Copy		Delete	3	3	10	15.00		
		Edit		Copy		Delete	3	5	15	4.00		
		Edit		Copy		Delete	4	3	50	15.00		
		Edit		Copy		Delete	4	5	1	4.00		
		Edit		Copy		Delete	5	3	1	15.00		
		Edit		Copy		Delete	5	5	1	4.00		
		Edit		Copy		Delete	6	1	1	5.50		
		Edit		Copy		Delete	9	12	1	500.00		
		Edit		Copy		Delete	10	5	4960	4.00		
		Edit		Copy		Delete	11	5	2000	4.00		
		Edit		Copy		Delete	14	12	1	500.00		
		Edit		Copy		Delete	15	1	100	5.50		
		Edit		Copy		Delete	15	2	500	25.00		
			Check all	With selected		Edit		Copy		Delete		Export

4.2 Project Assumptions and Scope Simplification

Removal of Import/Procurement Part: In the initial proposal, a module for importing medicines from foreign suppliers was included. However, this was later removed to focus mainly on the Sales and Inventory management. Managing foreign suppliers and purchase orders would have made the database relationships too complex for this phase.

Protecting Business History (Deletion Policy)

A major decision was made not to use the "ON DELETE CASCADE" feature for the Client and Medicine_Stock tables.

The Reason: If a client or a medicine is deleted from the system, using "CASCADE" would automatically delete all their past sales orders as well. This would destroy the company's financial history and past transaction records.

The Solution: Instead of automatic deletion, the system requires a "Manual Check." The database only allows a record to be deleted if there are no linked orders. This ensures that important business history is kept safe for future audits and reports.

Stock Accuracy and Automation

To ensure the inventory levels are always correct, the system was designed with automated stock updates.

Automatic Updates: When a Sales Order is "Approved" by the admin, the system automatically reduces the stock quantity.

Error Prevention: If an order is "Cancelled," the system automatically adds the items back to the inventory. This automation was implemented to prevent human errors and to make sure the medicine count in the system matches the physical stock.

4.3 Evaluation of Functional Scope

The final implementation was evaluated to check if the initial project objectives were met. It was concluded that the core functional scope of the system was successfully achieved.

- **User Management:** The system was developed to allow Hospitals and Pharmacies to register and log in. This functionality was fully implemented, and users were able to manage their profiles successfully.
- **Inventory Control:** The management of medicine stocks, including brand names, dosages, and expiry dates, was achieved. The Administrator was given the ability to update stock levels through the backend.
- **Sales Ordering:** The main goal of allowing clients to place orders and track their status (Pending, Approved, etc.) was fully realized.

- **Data Accuracy:** The system was designed to automatically update stock levels when an order was approved or cancelled. This ensured that the data remained accurate without manual intervention.

Conclusion: In conclusion, the project successfully satisfied all the critical requirements needed for a Pharmaceutical Distribution System. The implemented features provide a solid foundation for managing clients, stock, and orders effectively.

4.4 Team Individual's contribution

Project Proposal - All members

EERD - All members

Relational schema – All members

Implementation –

- Database – All members
- A - Backend database connection – AS2023631 - S A C HANSIKA
- B - Client login and dashboard – AS2023414 - G M A I Liyanage
- C – Client register - AS2023417 - G K LADDUWAHETTI
- D - Client place orders - AS2023414 - G M A I Liyanage
- E - Client view previous orders - AS2023475 - I M RATHNAYAKE
- F - Client update account - AS2023531 - K V S P WICKRAMARACHCHI
- G - Admin login - AS2023414 - G M A I Liyanage
- H- Admin manage medicine - AS2023608 - M W WIJewardana
- I - Admin client order management - AS2023633 - A D N Methmal
- J – Admin registered client management – AS2023631 – S A C HANSIKA
- K – Admin login business analytics and management – AS2023631 – S A C HANSIKA

AS2023531- K V S P WICKRAMARACHCHI	
AS2023475-I M RATHNAYAKE	
AS2023608-M W WIJewardana	
AS2023414- G M A I LIYANAGE	
AS2023633-A D N METHMAL	
AS2023417-G K LADDUWAHETTI	
AS2023631-S A C HANSIKA	